

BE AIML — Operating Systems

Mid-Term Exam Answers

Class Test I + Class Test II — Complete Solutions

University College of Engineering (A) | Osmania University, Hyderabad-07

Silberschatz, Galvin & Gagne — Operating System Concepts (9th Ed.)

Topic	Content
Class Test-I	Zombie/Orphan Process, Dual Mode, Multiprogramming, Graceful Degradation
CPU Scheduling	FCFS, SJF, Priority, Round Robin — Gantt charts + TAT + Waiting Time
OS Roles	Government / Resource Allocator / Control Program + Process Lifecycle
IPC & Threads	Shared Memory vs Message Passing + User vs Kernel Threads
Class Test-II	Deadlock conditions, Logic Bomb, Protection & Security, Memory Allocation
Banker's Algorithm	Safe state check, P1 request (1,0,2), P4 request (3,3,0) — fully solved
Page Replacement	Optimal & LRU for reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Security & Memory	Public Key Encryption, Access Matrix, Paging Technique

Class Test-I | Operating Systems

B.E V-Semester | CSE/AIML WP | Osmania University

Max Marks: 30 | Part-A: 5×2=10 | Part-B: 2×10=20 | Date: 23-03-2026

PART A — Short Answer Questions (5 × 2 = 10 Marks)

Answer

Q1. Parent P terminates while child C is still running — Will C immediately become a Zombie?

Answer: NO — C will NOT become a Zombie. It becomes an ORPHAN process.

What is a Zombie Process?

A zombie process is a process that has FINISHED execution but still has an entry in the process table, because its parent has not yet called wait() to collect its exit status. The zombie occupies no CPU or memory — only a PID and exit status slot.

What is an Orphan Process?

An orphan process is a process whose parent has TERMINATED while the child is STILL RUNNING. The operating system re-parents the orphan to the init process (PID 1) or systemd, which periodically calls wait() to clean up.

Why C does NOT become a Zombie:

- A zombie requires the child to finish FIRST (child terminates → parent has not called wait())
- Here, P (parent) terminates FIRST while C (child) is still RUNNING
- Since C is still running, it cannot be a zombie (which requires the process to have exited)
- Instead, C becomes an orphan; init adopts it and waits for C when C eventually finishes

Timeline: P forks C → both running P terminates → C is now ORPHAN (not zombie) init (PID 1) adopts C C eventually terminates → init calls wait() → C removed cleanly (no zombie)

Answer

Q2. What is dual mode operation? Why is it essential?

Dual Mode Operation is a hardware-supported mechanism in which the CPU can operate in exactly two modes — User Mode and Kernel Mode (also called Supervisor/Privileged Mode).

Property	User Mode	Kernel Mode
Privilege level	Restricted — limited instructions	Full — all instructions allowed
Runs	User application programs	OS kernel, device drivers
Hardware access	Indirect only (via system calls)	Direct — can control all hardware
Protection	Cannot execute privileged instructions	Can execute ANY instruction
Mode bit	1 (user)	0 (kernel)

How it works:

Mode bit in hardware: Mode bit = 1 → User Mode (restricted) Mode bit = 0 → Kernel Mode (privileged) Mode switch: User → Kernel: via system call / interrupt / exception Kernel → User: when OS returns control to user process

Why it is ESSENTIAL:

- Prevents user programs from executing privileged instructions (e.g. I/O, modifying page tables)
- Protects the OS kernel from accidental or malicious modification by user programs
- Protects user processes from each other — prevents one process from corrupting another memory
- Ensures that critical OS operations (like memory management, I/O) are performed safely
- Without dual mode, a buggy user program could crash the entire system or steal data

Answer

Q3. Give the objectives of Multiprogramming and Time Sharing.

Multiprogramming:

Multiprogramming is a technique where the OS keeps several programs in memory simultaneously. When one process waits for I/O, the CPU switches to another. The goal is to maximise CPU utilization.

- Objective 1: Maximise CPU utilization — CPU is never idle as long as there are runnable processes
- Objective 2: Increase throughput — more jobs completed per unit time
- Objective 3: Efficient memory use — multiple programs reside in RAM, reducing load time
- Technique: When process P1 waits for I/O, CPU switches to P2; when P2 blocks, switches to P3, etc.

Time Sharing (Multitasking):

Time sharing is an extension of multiprogramming where the CPU switches between processes so frequently (using a time quantum/slice) that each user gets the illusion of having a dedicated processor.

- Objective 1: Interactive response — short response time for user commands (< 1 second typically)
- Objective 2: Support multiple simultaneous users — each user gets a fair share of CPU time
- Objective 3: CPU switches every time quantum (e.g. 10-100 ms); user notices no delay
- Objective 4: Cost sharing — many users share one expensive computer system

Analogy: Multiprogramming = cooking multiple dishes; switch to stirring soup while pasta boils
Time sharing = taking turns at a shared phone — each person gets 30 seconds

Answer

Q4. Define graceful degradation and high availability in clustered systems.

Graceful Degradation:

The ability of a system to continue providing service (at reduced capacity or performance) even when part of the hardware or software fails — rather than completely stopping.

- A cluster with 8 nodes continues operating with 6 nodes if 2 nodes fail
- Performance degrades proportionally rather than catastrophically
- Also called fault tolerant design — the system tolerates partial failure

High Availability:

High availability refers to a system design that ensures the system remains operational and accessible for the maximum possible time — minimizing downtime. Typically measured as uptime percentage.

Availability %	Downtime per year	Downtime per day
99% (2 nines)	~87.6 hours	~14.4 minutes
99.9% (3 nines)	~8.76 hours	~1.44 minutes
99.99% (4 nines)	~52.6 minutes	~8.6 seconds
99.999% (5 nines)	~5.26 minutes	~0.86 seconds

Systems that require these properties:

- **Medical systems:** Patient monitoring, ICU equipment, surgical robots — a failure could cost lives
- **Financial systems:** Banking, stock exchanges, ATM networks — outage means lost revenue
- **Telecommunications:** 911 emergency, cellular networks — unavailability is life-threatening
- **Air traffic control:** Must never go down — flight safety depends on continuous operation
- **E-commerce / cloud:** Amazon, Google — downtime costs millions of dollars per minute

PART B — Long Answer Questions (Any 2 × 10 = 20 Marks)

Long
Answer [10
marks]

Q5. CPU Scheduling — FCFS, SJF, Non-Preemptive Priority, Round Robin with Gantt Charts, TAT & Waiting Time

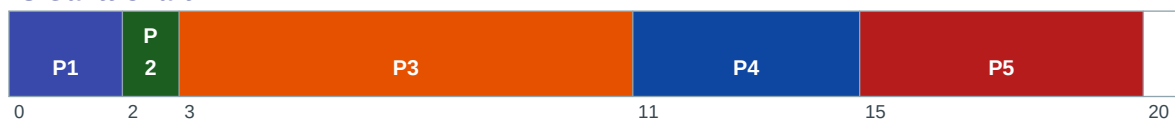
Given Process Data (all arrive at time 0, in order P1→P5):

Process	Burst Time (ms)	Priority (lower = higher priority)
P1	2	2
P2	1	1
P3	8	1
P4	4	4
P5	5	2

Algorithm 1 — FCFS (First Come First Served)

Execute processes in arrival order: P1 → P2 → P3 → P4 → P5. Non-preemptive.

FCFS Gantt Chart



Process	Arrival	Burst	Completion	Turnaround Time = CT-AT	Waiting Time = TAT-BT
P1	0	2	2	2-0 = 2	2-2 = 0
P2	0	1	3	3-0 = 3	3-1 = 2
P3	0	8	11	11-0 = 11	11-8 = 3
P4	0	4	15	15-0 = 15	15-4 = 11
P5	0	5	20	20-0 = 20	20-5 = 15

Average Waiting Time (FCFS) = $(0+2+3+11+15)/5 = 31/5 = 6.2$ ms
 Average Turnaround Time = $(2+3+11+15+20)/5 = 51/5 = 10.2$ ms

Algorithm 2 — SJF (Shortest Job First — Non-Preemptive)

Sort by burst time: P2(1) → P1(2) → P4(4) → P5(5) → P3(8). Execute shortest job first.

SJF Gantt Chart



Process	Burst	Completion	Turnaround Time	Waiting Time
P1	2	3	3-0 = 3	3-2 = 1

P2	1	1	1-0 = 1	1-1 = 0
P3	8	20	20-0 = 20	20-8 = 12
P4	4	7	7-0 = 7	7-4 = 3
P5	5	12	12-0 = 12	12-5 = 7

Average Waiting Time (SJF) = $(1+0+12+3+7)/5 = 23/5 = 4.6$ ms [OPTIMAL] Average Turnaround Time = $(3+1+20+7+12)/5 = 43/5 = 8.6$ ms

Algorithm 3 — Non-Preemptive Priority (lower number = higher priority)

Priority order: P2(1), P3(1), P1(2), P5(2), P4(4). Ties broken by arrival order.

Non-Preemptive Priority Gantt Chart



Process	Priority	Burst	Completion	Turnaround Time	Waiting Time
P1	2	2	11	11-0 = 11	11-2 = 9
P2	1	1	1	1-0 = 1	1-1 = 0
P3	1	8	9	9-0 = 9	9-8 = 1
P4	4	4	20	20-0 = 20	20-4 = 16
P5	2	5	16	16-0 = 16	16-5 = 11

Average Waiting Time (Priority) = $(9+0+1+16+11)/5 = 37/5 = 7.4$ ms Average Turnaround Time = $(11+1+9+20+16)/5 = 57/5 = 11.4$ ms Note: P4 (lowest priority) suffers starvation – waits 16ms!

Algorithm 4 — Round Robin (Time Quantum $q = 2$ ms)

Each process gets at most $q=2$ ms CPU time before being preempted. Queue: P1,P2,P3,P4,P5.

Step-by-step execution:

t=0-2: P1 runs (BT=2, done!) Queue after: P2,P3,P4,P5 t=2-3: P2 runs (BT=1, done!) Queue after: P3,P4,P5 t=3-5: P3 runs 2ms (rem=6) Queue after: P4,P5,P3 t=5-7: P4 runs 2ms (rem=2) Queue after: P5,P3,P4 t=7-9: P5 runs 2ms (rem=3) Queue after: P3,P4,P5 t=9-11: P3 runs 2ms (rem=4) Queue after: P4,P5,P3 t=11-13: P4 runs 2ms (rem=0, done!) Queue after: P5,P3 t=13-15: P5 runs 2ms (rem=1) Queue after: P3,P5 t=15-17: P3 runs 2ms (rem=2) Queue after: P5,P3 t=17-18: P5 runs 1ms (rem=0, done!) Queue after: P3 t=18-20: P3 runs 2ms (rem=0, done!)

Round Robin ($q=2$) Gantt Chart



Process	Burst	Completion	Turnaround Time	Waiting Time
P1	2	2	$2-0 = 2$	$2-2 = 0$
P2	1	3	$3-0 = 3$	$3-1 = 2$
P3	8	20	$20-0 = 20$	$20-8 = 12$
P4	4	13	$13-0 = 13$	$13-4 = 9$
P5	5	18	$18-0 = 18$	$18-5 = 13$

Average Waiting Time (RR $q=2$) = $(0+2+12+9+13)/5 = 36/5 = 7.2$ ms
Average Turnaround Time = $(2+3+20+13+18)/5 = 56/5 = 11.2$ ms

Comparison Summary:

Algorithm	Avg Waiting Time	Avg TAT	Best for
FCFS	6.2 ms	10.2 ms	Batch systems — simple, no starvation
SJF	4.6 ms (OPTIMAL)	8.6 ms	Minimum average WT — best if burst known
Priority	7.4 ms	11.4 ms	Real-time — important tasks first (starvation risk)
RR ($q=2$)	7.2 ms	11.2 ms	Interactive systems — fair, no starvation

Q7. OS as Government + Resource Allocator + Control Program & Process Lifecycle

(a) OS viewed as Government, Resource Allocator and Control Program

1. OS as a Government:

Just like a government, the OS itself does not perform useful work directly. Instead, it sets policies and provides a framework within which all activity occurs.

- A government makes laws → OS makes rules (policies) about resource use
- A government has departments (defense, finance) → OS has subsystems (memory mgr, file system, scheduler)
- A government provides services to citizens → OS provides services (system calls) to processes
- A government taxes and distributes wealth → OS allocates CPU time, memory, I/O to processes
- A government enforces laws via police → OS enforces protection via dual mode and memory protection

2. OS as a Resource Allocator:

A computer system has many resources that multiple programs may need simultaneously: CPU cycles, memory space, file storage, I/O devices, network bandwidth. The OS acts as a manager of all these resources.

- Decides WHO gets resources (process scheduling, memory allocation)
- Decides HOW MUCH each process gets (time quantum, memory pages)
- Decides WHEN to give/take resources (preemption, swapping)
- Ensures FAIR and EFFICIENT allocation — no single process starves others
- Examples: CPU scheduler, memory manager, disk I/O scheduler, network stack

3. OS as a Control Program:

The OS controls the execution of user programs to prevent errors and improper use of the computer. It manages I/O devices and ensures proper operation.

- Controls execution of all programs — starts, pauses, resumes, terminates processes
- Prevents unauthorized access to hardware (dual mode, privileged instructions)
- Controls I/O devices — provides device drivers, manages interrupts
- Catches errors: division by zero, illegal memory access → terminates offending process
- Prevents deadlocks, detects/recovers from system failures

(b) Life Cycle of a Process (Process States)

State	Description	Entry condition	Exit condition
New	Process being created	fork() / CreateProcess()	Admitted to ready queue
Ready	In memory, waiting for CPU	Admitted from New or I/O complete	Selected by scheduler
Running	Instructions being executed	Dispatcher assigns CPU	Time quantum expires / I/O request / terminates
Waiting	Waiting for I/O or event	Process requests I/O	I/O completion interrupt

Terminated	Finished execution	exit() called / error	OS removes from process table
-------------------	--------------------	-----------------------	-------------------------------

State transition diagram description:

New (admitted) Ready Ready (scheduler dispatch) Running Running (interrupt/time out) Ready Running (I/O or event wait) Waiting Waiting (I/O complete) Ready Running (exit/error) Terminated

Process Control Block (PCB) — data structure OS uses to store all process information:

- Process ID (PID), Process state (New/Ready/Running/Waiting/Terminated)
- Program counter (address of next instruction to execute)
- CPU registers (contents of all registers when process is not running)
- CPU scheduling info (priority, queue pointers)
- Memory management info (page tables, segment tables, base/limit registers)
- Accounting info (CPU used, clock time, time limits, account numbers)
- I/O status info (list of open files, list of I/O devices allocated to process)

Q8. Shared Memory vs Message Passing + User-level vs Kernel-level Threads

(a) Shared Memory vs Message Passing — IPC Comparison

Criterion	Shared Memory	Message Passing
Mechanism	Processes share a region of memory. Reads/writes directly to shared address space.	Processes communicate by sending/receiving explicit messages via OS.
Speed	FAST — once shared region established, no OS kernel intervention per access.	SLOWER — every message involves a system call (kernel mode switch).
Ease of implementation	HARDER — programmer must handle synchronization manually (mutex/semaphore).	EASIER — OS handles all synchronization; simple send()/receive() calls.
Scalability	Limited to same machine — shared memory cannot span physical machines.	Scales to distributed systems — processes can be on different computers (MPI, sockets).
Data inconsistency risk	HIGH — race conditions possible if multiple processes write without synchronization.	LOW — OS ensures message integrity; no shared state means no race conditions.
Typical use	High-performance single-machine IPC (databases, multimedia).	Distributed systems, microservices, client-server (MPI, POSIX MQ).
OS involvement	Only at setup (shmget/mmap); minimal thereafter.	Every send()/receive() involves a system call.

(b) User-level Threads vs Kernel-level Threads

Property	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Managed by	User-space thread library (POSIX, Java)	OS kernel directly
System call needed?	No — context switch in user space	Yes — each switch involves a system call
Speed	Fast context switching	Slower due to kernel mode switches
Blocking	If one thread blocks (I/O), ALL threads in process block	One thread blocks; other threads in same process continue
Multiprocessor use	Cannot use multiple CPUs in parallel	Can run threads on separate CPUs — true parallelism
OS awareness	Kernel sees only one process	Kernel manages each thread individually
Examples	POSIX pthreads (user mode), GNU pth	Windows threads, Linux NPTL

Three Threading Models:

1. Many-to-One (Many ULT → 1 KLT):

- Many user threads mapped to a single kernel thread

- Thread management done entirely by user-space library — very fast
- PROBLEM: If one thread makes a blocking system call, the ENTIRE process blocks
- PROBLEM: Cannot run in parallel on multiprocessor systems
- Example: Green threads (early Java), GNU Portable Threads

2. One-to-One (Each ULT → 1 KLT):

- Each user thread maps to exactly one kernel thread
- Blocking one thread does NOT block other threads — each has its own kernel thread
- True parallelism — threads can run on different CPU cores simultaneously
- DISADVANTAGE: Creating a user thread creates a kernel thread — overhead; limited by OS
- Examples: Windows Vista/7/10, Linux (NPTL/pthreads), Solaris 9+

3. Many-to-Many (Many ULT → Many KLT):

- Many user threads multiplexed onto an equal or smaller number of kernel threads
- Combines advantages of both — programmer can create as many ULTs as needed
- OS creates kernel threads as needed; if one ULT blocks, kernel can schedule another ULT
- Supports true parallelism without the limitation of 1:1 overhead
- Examples: Solaris <9, Windows NT with fiber package, IRIX

Class Test-II | Operating Systems

B.E V-Semester | CSE/AIML WP | Osmania University

Max Marks: 30 | Part-A: 5×2=10 | Part-B: 2×10=20 | Date: 04-05-2026

PART A — Short Answer Questions (5 × 2 = 10 Marks)

Answer

Q1. What are the four conditions that a deadlock can arise?

A deadlock can arise if and only if all FOUR of the following conditions (Coffman Conditions, 1971) hold simultaneously:

Condition	Definition	Example
1. Mutual Exclusion	At least one resource must be held in a non-sharable mode — only one process can use it at a time.	A printer can only be used by one process at a time
2. Hold and Wait	A process holds at least one resource AND is waiting to acquire additional resources held by other processes.	P1 holds Resource A and is waiting for Resource B
3. No Preemption	Resources cannot be forcibly taken away from a process. A resource can only be released voluntarily by the holding process.	Cannot take a locked file from P1 — P1 must release it voluntarily
4. Circular Wait	A circular chain of processes exists: P1 waits for P2, P2 waits for P3, ..., Pn waits for P1.	P1→waits for→P2→waits for→P3→waits for→P1

Deadlock prevention: Negate any ONE of the four conditions. Mutual exclusion: cannot prevent for non-sharable resources Hold & Wait: require process to request ALL resources at once No Preemption: allow OS to preempt and re-allocate resources Circular Wait: impose total ordering on resource types

Answer

Q2. What is a logic bomb? How it may manifest itself?

A **logic bomb** is a piece of malicious code secretly inserted into a software system or program that lies dormant and inactive until a specific triggering condition or event occurs, at which point it executes its payload (destructive or malicious action).

How it manifests:

- **Date/time trigger:** Code executes on a specific date (e.g., deletes files on April 1st, or every Friday the 13th)
- **Login trigger:** Activates when a specific username logs in (e.g., when dismissed employee's account is deleted — as revenge)
- **Counter trigger:** Activates after a program has been run N times (e.g., after 100 uses without payment)
- **File presence/absence:** Triggers when a specific file exists or is deleted
- **Network event:** Triggers upon network connection to a specific server

Real-world example:

A disgruntled employee embeds code in payroll software: IF (employee_name NOT IN payroll_list) THEN DELETE ALL FROM database END IF As long as the employee stays, nothing happens. The day they are fired (removed from payroll), the bomb triggers.

Answer

Q3. Define Protection & Security.

Aspect	Protection	Security
Definition	Mechanisms to control access of processes and users to OS-defined resources	Measures to defend the entire system against external/internal attacks
Scope	Internal to the system — between processes and resources	External and internal threats — attackers, malware, unauthorized users
Goal	Ensure each process accesses only what it is allowed to	Prevent unauthorized access, data theft, system destruction
Examples	File permissions (rwx), memory bounds, privileged instructions	Passwords, encryption, firewalls, antivirus, VPN
OS support	Access control lists, mode bits, rings of protection	Authentication, audit logs, encryption, intrusion detection
Mechanism type	Primarily hardware + OS policy enforcement	Combination of hardware, software, procedural controls

Protection: Is process P allowed to access resource R? (mechanism) Security: Is the entity claiming to be user X actually user X? (authentication)
"Protection is a mechanism; Security is a policy enforced via many mechanisms."

Answer

Q4. Mention best fit, worst fit and first fit allocation strategies.

These are dynamic memory allocation strategies for assigning free memory holes to processes:

Strategy	Algorithm	Advantage	Disadvantage
First Fit	Allocate the FIRST hole (from beginning of list) that is large enough	Fastest — scan stops at first match. Less overhead.	Leaves small unusable fragments near start of memory
Best Fit	Allocate the SMALLEST hole that is large enough. Must scan entire list.	Minimizes wasted space per allocation — tightest fit.	Slowest — must scan all holes. Produces tiny unusable fragments throughout.
Worst Fit	Allocate the LARGEST available hole. Must scan entire list.	Leaves large remaining fragments — more useful leftovers.	Slowest — scans all. Fragments large holes that could serve big processes.

Example: Holes available: 100K, 500K, 200K, 300K, 600K Process requests 212K: First Fit → allocate from 500K hole (first one large enough) Remaining fragment: 288K in that hole Best Fit → allocate from 300K hole (smallest that fits 212K) Remaining fragment: 88K (smallest leftover) Worst Fit → allocate from 600K hole (largest hole) Remaining fragment: 388K (largest leftover for future use)

PART B — Long Answer Questions (Any 2 × 10 = 20 Marks)

Long
Answer [10
marks]

Q5. Banker's Algorithm — Safe State Check + P1 Request (1,0,2) + P4 Request (3,3,0)

Given: 5 processes (P0-P4), 3 resource types: A(10), B(5), C(7)

Step 1 — Setup tables:

Process	Alloc A	Alloc B	Alloc C	Max A	Max B	Max C	Need A	Need B	Need C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

Total Resources: A=10, B=5, C=7 Total Allocated: A=(0+2+3+2+0)=7,
B=(1+0+0+1+0)=2, C=(0+0+2+1+2)=5 Available = Total - Allocated = A=(10-7)=3,
B=(5-2)=3, C=(7-5)=2 Need[i][j] = Max[i][j] - Allocation[i][j]

Step 2 — Safety Algorithm (Work = Available = [3,3,2]):

Work=[3,3,2], Finish=[F,F,F,F,F] Iteration 1: Scan for process where Need ≤ Work: P0: Need=[7,4,3] ≤ [3,3,2]? No (7>3) P1: Need=[1,2,2] ≤ [3,3,2]? YES! (1≤3, 2≤3, 2≤2) Execute P1: Work = [3+1, 3+0, 2+2] = [4,3,4], Finish[1]=True Iteration 2: Scan remaining (P0,P2,P3,P4): P0: Need=[7,4,3] ≤ [4,3,4]? No (7>4) P2: Need=[6,0,0] ≤ [4,3,4]? No (6>4) P3: Need=[0,1,1] ≤ [4,3,4]? YES! (0≤4, 1≤3, 1≤4) Execute P3: Work = [4+0, 3+1, 4+1] = [4,4,5], Finish[3]=True Iteration 3: P0: Need=[7,4,3] ≤ [4,4,5]? YES! (7≤4, 4≤4, 3≤5) Execute P0: Work = [4+7, 4+4, 5+3] = [11,8,8], Finish[0]=True Iteration 4: P2: Need=[6,0,0] ≤ [11,8,8]? YES! (6≤11, 0≤8, 0≤8) Execute P2: Work = [11+6, 8+0, 8+0] = [17,8,8], Finish[2]=True Iteration 5: P4: Need=[4,3,1] ≤ [17,8,8]? YES! (4≤17, 3≤8, 1≤8) Execute P4: Work = [17+4, 8+3, 8+1] = [21,11,9], Finish[4]=True All Finish = True! SAFE SEQUENCE: P1 → P3 → P0 → P2 → P4 SYSTEM IS IN A SAFE STATE.

Part (a) — P1 requests (1, 0, 2) (1 instance of A, 0 of B, 2 of C):

Request1 = [1,0,2] Check 1: Request ≤ Need[P1]: [1,0,2] ≤ [1,2,2]? YES (1≤1, 0≤2, 2≤2) Check 2: Request ≤ Available: [1,0,2] ≤ [3,3,2]? YES (1≤3, 0≤3, 2≤2) Tentatively grant — update state: Available = [3-1, 3-0, 2-2] = [2, 3, 0] Allocation[P1] = [2+1, 0+0, 0+2] = [3, 0, 2] Need[P1] = [1-1, 2-0, 2-2] = [0, 2, 0] Run Safety Algorithm with Work=[2,3,0]: P1: Need=[0,2,0] ≤ [2,3,0]? YES → Work=[5,3,2] P3: Need=[0,1,1] ≤ [5,3,2]? YES → Work=[7,4,3] P0: Need=[7,4,3] ≤ [7,4,3]? YES → Work=[7,5,3] P2: Need=[6,0,0] ≤ [7,5,3]? YES → Work=[10,5,3] P4: Need=[4,3,1] ≤ [10,5,3]? YES → Work=[10,5,7] SAFE SEQUENCE: P1→P3→P0→P2→P4 CONCLUSION: Request by P1 for (1,0,2) CAN BE GRANTED IMMEDIATELY.

Part (b) — P4 requests (3, 3, 0):

Request₄ = [3,3,0] Check 1: Request ≤ Need[P₄]: [3,3,0] ≤ [4,3,1]? YES (3≤4, 3≤3, 0≤1)
 Check 2: Request ≤ Available: [3,3,0] ≤ [3,3,2]? YES (3≤3, 3≤3, 0≤2) Tentatively grant —
 update state: Available = [3-3, 3-3, 2-0] = [0, 0, 2] Allocation[P₄] = [0+3, 0+3, 2+0] = [3, 3, 2]
 Need[P₄] = [4-3, 3-3, 1-0] = [1, 0, 1] Run Safety Algorithm with Work=[0,0,2]: P₀: Need=[7,4,3]
 ≤ [0,0,2]? No (7>0) P₁: Need=[1,2,2] ≤ [0,0,2]? No (1>0) P₂: Need=[6,0,0] ≤ [0,0,2]? No
 (6>0) P₃: Need=[0,1,1] ≤ [0,0,2]? No (1>0 for B) P₄: Need=[1,0,1] ≤ [0,0,2]? No (1>0 for A)
 NO PROCESS CAN PROCEED! Finish still all False. UNSAFE STATE DETECTED!
 CONCLUSION: Request by P₄ for (3,3,0) CANNOT BE GRANTED. System must wait or P₄
 must reduce its request.

Q6. Optimal & LRU Page Replacement + Public Key Encryption

(a) Page Replacement — Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 | Frames = 3

Optimal Page Replacement Algorithm — Replace page used farthest in future:

Rule: When a page fault occurs, replace the page whose next use is farthest in the future. If a page is never used again, it is the first candidate for replacement.

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
F1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
F2	-	0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
F3	-	-	1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
Fault	F	F	F	F	-	F	-	F	-	-	F	-	-	F	-	-	-	F	-	-

Optimal Page Faults: at positions 1,2,3,4,6,8,11,14,18 = 9 faults (Optimal gives the MINIMUM possible page faults – theoretical best case)

LRU (Least Recently Used) Page Replacement — Replace page not used for longest time:

Rule: When a page fault occurs, replace the page that was used least recently (the one that has not been accessed for the longest time in the past).

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
F1	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	7	7	7
F2	-	0	0	0	0	0	0	0	2	2	2	2	2	2	2	0	0	0	0	0
F3	-	-	1	1	1	3	3	3	3	3	3	3	3	3	3	3	1	1	1	1
Fault	F	F	F	F	-	F	-	F	F	F	F	-	-	F	-	F	-	F	-	-

LRU Page Faults: at positions 1,2,3,4,6,8,9,10,11,14,16,18 = 12 faults
Comparison: Optimal: 9 faults (minimum possible – requires future knowledge)
LRU: 12 faults (practical, uses past to approximate future)

(b) Encryption — Process + Public Key Encryption

What is Encryption?

Encryption is the process of transforming plaintext (readable data) into ciphertext (unreadable form) using a mathematical algorithm and a key, so that only authorised parties with the correct key can decrypt and read the original data.

Encryption: plaintext + key → [encryption algorithm] → ciphertext
Decryption: ciphertext + key → [decryption algorithm] → plaintext

Symmetric Encryption: Same key used for encryption and decryption. Fast but key distribution is a problem (AES, DES).

Public Key Encryption (Asymmetric):

Public key encryption uses a mathematically linked key PAIR: a PUBLIC key (freely distributed, known to everyone) and a PRIVATE key (kept secret by the owner). What one key encrypts, only the other can decrypt.

Property	Detail
Key pair	Public key (shared openly) + Private key (kept secret)
Encryption	Sender encrypts with receiver's PUBLIC key
Decryption	Only receiver can decrypt — using their PRIVATE key
Security basis	Computational difficulty: factoring large primes (RSA), discrete logarithm (ECC)
Algorithm examples	RSA (Rivest-Shamir-Adleman), ECC (Elliptic Curve), Diffie-Hellman
Advantage	No need to share a secret key — public key can be published openly
Disadvantage	100x-1000x slower than symmetric — used for key exchange, signatures, not bulk data
Digital signatures	Sender signs with PRIVATE key; anyone can verify with PUBLIC key

Example: Alice wants to send a secure message to Bob: 1. Bob publishes his PUBLIC KEY to the world 2. Alice encrypts message with Bob's PUBLIC KEY 3. Ciphertext travels over the internet 4. Only Bob can decrypt it with his PRIVATE KEY Even if an attacker intercepts the ciphertext and knows Bob's public key, they cannot decrypt it without Bob's private key.

Q7. Access Matrix Implementation + Paging Technique

(a) Implementation of Access Matrix

The access matrix is a model for protection that expresses allowed operations. Rows = domains (users/processes), Columns = objects (files, devices, processes). Entry $\text{access}[i][j]$ = set of allowed operations domain i can perform on object j .

Example Access Matrix:

Domain	File F1	File F2	File F3	Printer	Process P1
D1 (User A)	read, write	read	—	print	wakeup
D2 (User B)	read	read, write	execute	—	—
D3 (System)	read, write, exec	read, write	read, write	print	all

Three implementation approaches:

1. Global Table — Store all triples (domain, object, rights) in one table:

- Simplest implementation — one table with rows for each (Domain, Object, Rights) triple
- Disadvantage: Very large table — waste of memory; slow lookup
- Cannot group objects that all domains access the same way

2. Access Control Lists (ACLs) — Column-based decomposition:

- Each OBJECT has a list of (domain, rights) pairs
- File F1 ACL: [(D1: read,write), (D2: read), (D3: all)]
- Advantage: Easy to know who can access a file; easy to revoke access per object
- Disadvantage: Determining all rights of a domain requires scanning all ACLs
- Used in: UNIX/Linux (file permissions rwxrwxrwx), Windows NTFS

3. Capability Lists — Row-based decomposition:

- Each DOMAIN has a list of (object, rights) pairs called capabilities
- Domain D1 capabilities: [(F1: read,write), (F2: read), (Printer: print)]
- A capability = an unforgeable token granting rights to an object
- Advantage: Fast to determine what a process can do; easy to transfer capabilities
- Disadvantage: Hard to revoke capabilities once granted; capabilities must be protected
- Used in: Mach, Hydra OS, KeyKOS

(b) Paging Technique

Paging is a memory management scheme that eliminates external fragmentation by dividing both physical memory and logical address space into fixed-size blocks called FRAMES and PAGES respectively.

Key terms:

Term	Definition	Typical size
Frame	Fixed-size block of PHYSICAL memory	4KB, 8KB, or 16KB

Page	Fixed-size block of LOGICAL (virtual) address space	Same as frame size
Page table	OS data structure mapping each logical page number to physical frame number	One entry per page
Page number (p)	Upper bits of logical address — index into page table	Depends on address space size
Page offset (d)	Lower bits of logical address — offset within the page/frame	$\log_2(\text{page size})$ bits

Address translation:

Logical address = (page number p, offset d) Physical address = (frame number f, offset d)
 Translation steps: 1. Extract page number p from upper bits of logical address
 2. Look up page table: frame number $f = \text{page_table}[p]$ 3. Physical address = $(f * \text{page_size}) + d$
 Example: Page size = 4KB = 4096 bytes (12 bits for offset) Logical address 0x3A7F: $p = 0x3A7F \gg 12 = 3$, $d = 0x7F = 127$
 If $\text{page_table}[3] = 7$ (frame 7): Physical address = $7 * 4096 + 127 = 28672 + 127 = 28799$

Advantages and Disadvantages of Paging:

Advantages	Disadvantages
No external fragmentation — any free frame can be used for any page	Internal fragmentation — last page may not fill its frame completely
Non-contiguous allocation — process pages can be scattered in physical memory	Page table overhead — large address spaces need huge page tables
Easy to share pages — two processes can point to same frame (shared libraries)	TLB miss penalty — page table access takes time (requires TLB for speed)
Supports virtual memory — pages can be on disk (demand paging)	Memory access time doubles without TLB — must access page table first

Quick Formula Reference — Both Mid-Terms

Concept	Formula / Definition
Turnaround Time	$TAT = \text{Completion Time} - \text{Arrival Time}$
Waiting Time	$WT = \text{Turnaround Time} - \text{Burst Time}$
Response Time	$RT = \text{Time of first CPU response} - \text{Arrival Time}$
CPU Utilization	$CPU\% = (\text{Busy time} / \text{Total time}) \times 100$
Throughput	Number of processes completed per unit time
Need[i][j] (Banker)	$Need[i][j] = Max[i][j] - Allocation[i][j]$
Available resources	$Available = Total - \sum(Allocation \text{ for all processes})$
Safe state (Banker)	System is safe if a safe sequence $P_1, P_2, \dots, P_n$ exists
Page table entry	$Physical \text{ address} = (\text{frame\#} \times \text{page_size}) + \text{offset}$
Effective Access Time	$EAT = \text{hit_ratio} \times TLB_time + (1 - \text{hit_ratio}) \times (TLB_time + \text{mem_time})$
FCFS avg WT	Process i WT = sum of burst times of processes before i
SJF property	SJF gives minimum average waiting time for non-preemptive algorithms
RR time quantum	Smaller q = more responsive but more context switch overhead